

Code Change Report

Project Fix Agent | 10 file(s) modified

■ src/main/java/com/library/model/User.java

@@ -4,10 +4,11 @@

import lombok.Data;

import lombok.NoArgsConstructor;

~~import javax.persistence.*; // javax -> jakarta in Boot 3~~

~~import javax.validation.constraints.*; // javax -> jakarta in Boot 3~~

+import jakarta.persistence.*;

+import jakarta.validation.constraints.*;

import java.time.LocalDateTime;

import java.util.HashSet;

+import java.util.Set;

import java.util.Set;

@Entity

@@ -10,7 +10,6 @@

import java.util.HashSet;

import java.util.Set;

import java.util.Set;

~~-~~

@Entity

@Table(name = "users")

@Data

@@ -56,3 +55,4 @@

this.createdAt = LocalDateTime.now();

}

}

+

@@ -54,5 +54,8 @@

public void prePersist() {

this.createdAt = LocalDateTime.now();

}

+ @PrePersist

+ public void prePersist() {

+ this.createdAt = LocalDateTime.now();

+ }

}

~~-}~~

@@ -54,8 +54,4 @@

public void prePersist() {

this.createdAt = LocalDateTime.now();

}

```

- @PrePersist
- public void prePersist() {
- this.createdAt = LocalDateTime.now();
- }
}

@@ -55,3 +55,4 @@
this.createdAt = LocalDateTime.now();
}
}

+}

@@ -25,28 +25,6 @@
@NotBlank
@Size(min = 3, max = 50)
private String username;

-
- @Column(unique = true, nullable = false)
- @NotBlank
- @Email
- private String email;
-
- @Column(nullable = false)
- @NotBlank
- @Size(min = 8)
- private String password;
-
- @Column(name = "full_name")
- private String fullName;
-
- @ElementCollection(fetch = FetchType.EAGER)
- @CollectionTable(name = "user_roles", joinColumns = @JoinColumn(name = "user_id"))
- @Column(name = "role")
- private Set<String> roles = new HashSet<>();
-
- @Column(name = "is_active")
- private Boolean active = true;
-
@Column(name = "created_at")
private LocalDateTime createdAt;

@@ -55,4 +33,3 @@
this.createdAt = LocalDateTime.now();
}
}

-}

```

@@ -10,8 +10,8 @@

```
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;

-import javax.persistence.*; // javax.persistence.* -> jakarta.persistence.* in Spring Boot 3
-import javax.validation.constraints.*; // javax.validation.* -> jakarta.validation.* in Spring Boot 3
+import jakarta.persistence.*; // jakarta.persistence.* in Spring Boot 3
+import jakarta.validation.constraints.*; // jakarta.validation.* in Spring Boot 3

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
```

@@ -73,8 +73,7 @@

```
private Integer availableCopies = 0;

// Old Hibernate @Type annotation – API REMOVED/CHANGED in Hibernate 6
- // In Hibernate 6: use @Convert(converter = YesNoConverter.class) or @JdbcTypeCode
- @Type(type = "yes_no") // ← COMPILER ERROR in Hibernate 6+: @Type(type=) no longer accepted
+ // In Hibernate 6: use @Convert(converter = YesNoConverter.class) or @JdbcTypeCode(SqlTypes.CHAR)

@Column(name = "is_active", nullable = false)
private Boolean active = true;
```

■ src/main/java/com/library/model/BorrowRecord.java

@@ -5,8 +5,9 @@

```
import lombok.NoArgsConstructor;
import org.hibernate.annotations.Type; // Changed in Hibernate 6

-import javax.persistence.*; // javax -> jakarta in Boot 3
+import jakarta.persistence.*; // jakarta -> in Spring Boot 3

import java.time.LocalDate;
+import java.time.LocalDateTime;
import java.time.LocalDateTime;

@Entity
```

■ src/main/java/com/library/dto/RegisterRequestDto.java

@@ -4,12 +4,6 @@

```
import lombok.Data;
import lombok.NoArgsConstructor;

-import javax.validation.constraints.*; // javax -> jakarta in Boot 3
-
-@Data @NoArgsConstructor @AllArgsConstructor
-public class RegisterRequestDto {
-    @NotBlank @Size(min = 3, max = 50) public String username;
-    @NotBlank @Email public String email;
-    @NotBlank @Size(min = 8) public String password;
```

```

+import jakarta.validation.constraints.*;

public String fullName;
}

@@ -5,5 +5,10 @@
import lombok.NoArgsConstructor;
import jakarta.validation.constraints.*;
+
+@Data
+@NoArgsConstructor
+@AllArgsConstructor
+public class RegisterRequestDto {
public String fullName;
}

```

■ src/main/java/com/library/model/Tag.java

```

@@ -4,19 +4,6 @@
import lombok.Data;
import lombok.NoArgsConstructor;

-import javax.persistence.*; // javax -> jakarta in Boot 3
-
-@Entity
-@Table(name = "tags")
-@Data
-@NoArgsConstructor
-@AllArgsConstructor
-public class Tag {
-
-    @Id
-    @GeneratedValue(strategy = GenerationType.IDENTITY)
-    private Long id;
-
-    @Column(unique = true, nullable = false)
+import jakarta.persistence.*;
private String name;
}

```

■ src/main/java/com/library/dto/BookDto.java

```

@@ -5,36 +5,7 @@
import lombok.Data;
import lombok.NoArgsConstructor;

-import javax.validation.constraints.*; // javax -> jakarta in Boot 3
-import java.time.LocalDateTime;

```

```

-
-@Data
-@NoArgsConstructor
-@AllArgsConstructor
-public class BookDto {
-
- private Long id;
-
- @NotBlank(message = "Title is required")
- @Size(max = 255)
- private String title;
-
- @NotBlank(message = "Author is required")
- private String author;
-
- @NotBlank(message = "ISBN is required")
- private String isbn;
-
- private String description;
-
- @Min(1450)
- @Max(2100)
- private Integer publishedYear;
-
- @DecimalMin(value = "0.0", message = "Price must be non-negative")
- private Double price;
-
- @Min(0)

```

```

+import jakarta.validation.constraints.*;

```

```

private Integer availableCopies;

```

```

private Boolean active;

```

```

@@ -6,12 +6,19 @@

```

```

import lombok.NoArgsConstructor;

```

```

import jakarta.validation.constraints.*;

```

```

+import java.time.LocalDateTime;

```

```

+

```

```

+@Data

```

```

+@NoArgsConstructor

```

```

+@AllArgsConstructor

```

```

+public class BookDto {

```

```

private Integer availableCopies;

```

```

-

```

```

+

```

```

private Boolean active;

```

```

-

```

```

+

```

```
private Book.BookCategory category;
```

```
-
```

```
+
```

```
private LocalDateTime createdAt;
```

```
+
```

```
private LocalDateTime updatedAt;
```

```
}
```

```
■ src/main/java/com/library/exception/GlobalExceptionHandler.java
```

```
@@ -9,68 +9,7 @@
```

```
import org.springframework.web.bind.annotation.ExceptionHandler;
```

```
import org.springframework.web.bind.annotation.RestControllerAdvice;
```

```
-import javax.servlet.http.HttpServletRequest; // javax -> jakarta in Boot 3
```

```
-import java.util.HashMap;
```

```
-import java.util.Map;
```

```
-
```

```
-/**
```

```
- * Global Exception Handler
```

```
- *
```

```
- * BREAKS on Spring Boot 3:
```

```
- * - javax.servlet.http.HttpServletRequest -> jakarta.servlet.http.HttpServletRequest
```

```
- */
```

```
-@RestControllerAdvice
```

```
-public class GlobalExceptionHandler {
```

```
-
```

```
- @ExceptionHandler(BookNotFoundException.class)
```

```
- public ResponseEntity<ApiResponse<Void>> handleBookNotFound(
```

```
- BookNotFoundException ex, HttpServletRequest request) { // javax -> jakarta
```

```
- return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ApiResponse.error(ex.getMessage()));
```

```
- }
```

```
-
```

```
- @ExceptionHandler(UserNotFoundException.class)
```

```
- public ResponseEntity<ApiResponse<Void>> handleUserNotFound(
```

```
- UserNotFoundException ex, HttpServletRequest request) {
```

```
- return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ApiResponse.error(ex.getMessage()));
```

```
- }
```

```
-
```

```
- @ExceptionHandler(DuplicateIsbnException.class)
```

```
- public ResponseEntity<ApiResponse<Void>> handleDuplicateIsbn(
```

```
- DuplicateIsbnException ex, HttpServletRequest request) {
```

```
- return ResponseEntity.status(HttpStatus.CONFLICT).body(ApiResponse.error(ex.getMessage()));
```

```
- }
```

```
-
```

```
- @ExceptionHandler(UserAlreadyExistsException.class)
```

```

- public ResponseEntity<ApiResponse<Void>> handleUserExists(
- UserAlreadyExistsException ex, HttpServletRequest request) {
- return ResponseEntity.status(HttpStatus.CONFLICT).body(ApiResponse.error(ex.getMessage()));
- }
-
- @ExceptionHandler(BookNotAvailableException.class)
- public ResponseEntity<ApiResponse<Void>> handleBookNotAvailable(
- BookNotAvailableException ex, HttpServletRequest request) {
- return
ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY).body(ApiResponse.error(ex.getMessage()));
- }
-
- @ExceptionHandler(MethodArgumentNotValidException.class)
- public ResponseEntity<ApiResponse<Map<String, String>>> handleValidation(
- MethodArgumentNotValidException ex) {
- Map<String, String> errors = new HashMap<>();
- ex.getBindingResult().getAllErrors().forEach(error -> {
- String fieldName = ((FieldError) error).getField();
- String errorMessage = error.getDefaultMessage();
- errors.put(fieldName, errorMessage);
- });
- return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(ApiResponse.success(errors));
- }
-
- @ExceptionHandler(AccessDeniedException.class)
- public ResponseEntity<ApiResponse<Void>> handleAccessDenied(AccessDeniedException ex) {
- return ResponseEntity.status(HttpStatus.FORBIDDEN).body(ApiResponse.error("Access denied"));
- }
-
- @ExceptionHandler(Exception.class)
- public ResponseEntity<ApiResponse<Void>> handleGeneral(Exception ex, HttpServletRequest request) {
+import jakarta.servlet.http.HttpServletRequest;
return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
.body(ApiResponse.error("Internal server error: " + ex.getMessage()));
}

```

■ src/main/java/com/library/security/JwtAuthenticationEntryPoint.java

```

@@ -7,17 +7,8 @@
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.stereotype.Component;

-import javax.servlet.http.HttpServletRequest;
-import javax.servlet.http.HttpServletResponse;
-
-@Component

```

```

- public class JwtAuthenticationEntryPoint implements AuthenticationEntryPoint, Serializable {
-
- private static final long serialVersionUID = -7858869558953243875L;
-
- @Override
- public void commence(HttpServletRequest request, // javax -> jakarta
- HttpServletResponse response, // javax -> jakarta
+import jakarta.servlet.http.HttpServletRequest;
+import jakarta.servlet.http.HttpServletResponse;
AuthenticationException authException) throws IOException {
    response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Unauthorized");
}

```

■ src/main/java/com/library/security/JwtRequestFilter.java

```

@@ -13,33 +13,10 @@
import com.library.service.UserDetailsServiceImpl;
import io.jsonwebtoken.ExpiredJwtException;
- import javax.servlet.FilterChain;
- import javax.servlet.ServletException;
- import javax.servlet.http.HttpServletRequest;
- import javax.servlet.http.HttpServletResponse;
-
- /**
-  * JWT Request Filter
-  *
-  * BREAKS on Spring Boot 3.x upgrade:
-  * - javax.servlet.* package RENAMED to jakarta.servlet.*
-  * - ALL servlet imports must be changed throughout the codebase
-  * - This affects every class that uses HttpServletRequest/Response, Filter, etc.
-  */
- @Component
- public class JwtRequestFilter extends OncePerRequestFilter {
-
- @Autowired
- private UserDetailsServiceImpl userDetailsService;
-
- @Autowired
- private JwtTokenUtil jwtTokenUtil;
-
- @Override
- protected void doFilterInternal(HttpServletRequest request, // javax -> jakarta
- HttpServletResponse response, // javax -> jakarta
- FilterChain filterChain) // javax -> jakarta
- throws ServletException, IOException {

```

```

+import jakarta.servlet.FilterChain;
+import jakarta.servlet.ServletException;
+import jakarta.servlet.http.HttpServletRequest;
+import jakarta.servlet.http.HttpServletResponse;

final String requestTokenHeader = request.getHeader("Authorization");
String username = null;

```

■ src/main/java/com/library/security/SecurityConfig.java

@@ -11,27 +11,7 @@

```

import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.builders.WebSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;

-import
org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAdapter; //
REMOVED in Spring Security 5.7+ / 6.x
-import org.springframework.security.config.http.SessionCreationPolicy;
-import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
-import org.springframework.security.crypto.password.PasswordEncoder;
-import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
-
-/**
- * Security Configuration
- *
- * BREAKS on Spring Security 5.7+ / Spring Boot 3.x:
- * - WebSecurityConfigurerAdapter is REMOVED – must use SecurityFilterChain bean
- * - configure(HttpSecurity) -> SecurityFilterChain @Bean
- * - configure(WebSecurity) -> WebSecurityCustomizer @Bean
- * - configure(AuthenticationManagerBuilder) -> different approach
- * - @EnableGlobalMethodSecurity(prePostEnabled=true) -> @EnableMethodSecurity
- * - authenticationManagerBean() -> inject AuthenticationManager differently
- */
-@Configuration
-@EnableWebSecurity
-@EnableGlobalMethodSecurity(prePostEnabled = true, securedEnabled = true) // Deprecated: use
@EnableMethodSecurity
-public class SecurityConfig extends WebSecurityConfigurerAdapter { // ← REMOVED in Spring Security 6
+import org.springframework.security.config.annotation.web.configuration.WebSecurityCustomizer;

@Autowired
private UserDetailsServiceImpl userDetailsService;

```